

REQ4 – Witch Mayhem of Potions

A. Potion Effect Implementation

Design 1: Abstract Potion Class with Subclasses and StatusEffect Objects

Pros	Cons
The potion itself (Potion) is separated from its effects (StatusEffect), ensuring distinct responsibilities and improving modularity.	Each New Effect Requires a StatusEffect Subclass: Creating a new effect (e.g., SpeedEffect) necessitates a new StatusEffect subclass, increasing the number of classes to manage.
Adding a new potion effect only requires a new Potion subclass and corresponding StatusEffect subclass, adhering to the Open-Closed Principle OCP.	
The same StatusEffect (e.g., PoisonEffect) can be triggered by different potions or items (if designed to allow this), promoting code reuse and DRY.	

Design 2: Hardcoded Effect Logic in Each Specific Potion Class

Pros	Cons
Direct for Simple Effects: For very simple effects (e.g., instant healing), hardcoding logic in the potion class is straightforward, reducing the need for additional classes.	Effect logic is bound to the potion class, limiting reuse in other contexts (e.g., non-potion items) and reducing modularity.
Minimal Initial Setup: Requires fewer classes and abstractions, allowing rapid prototyping for small systems with basic effects.	The potion class manages both identity and effect logic, mixing concerns and hindering maintainability.
	Similar effects (e.g. StrongHealingPotion) can lead to duplicated logic, violating DRY and increasing maintenance burden.

B. Potion Brewing System

Design 1: Modular Brewing via Pouch, BrewPotionAction and Potion classes

Pros	Cons
Pouch interconnects Brewing action and potions with Player class, while BrewPoitionAction executes brewing, ensuring modular and maintainable code.	Everytime a new Potion is added we need create more classes which lengthen the codebase.
Defining recipes as Status lists inside Potion classes enables easy modification and supports a dynamic, extensible crafting system.	Increase abstraction layer due to Pouch class.
Shared material validation and consumption logic in Pouch reduces duplication and adheres to DRY principles.	
New potions can be added without modifying existing code, aligning with the Open/Closed Principle.	

Design 2: Player Class Directly Handles All Brewing Logic with Hardcoded Item Names

Pros	Cons
Consolidating brewing logic in Player can be simple and quick for small systems with few potions.	Brewing logic is tightly bound to Player and specific items, limiting modularity and flexibility.
Allows rapid implementation during early development with basic brewing needs.	Adding new recipes requires modifying complex if-else structures, increasing error risk and maintenance overhead
	Material checks and consumption are repeated across branches, violating DRY principles.

C. Meat Ingredients Implementation

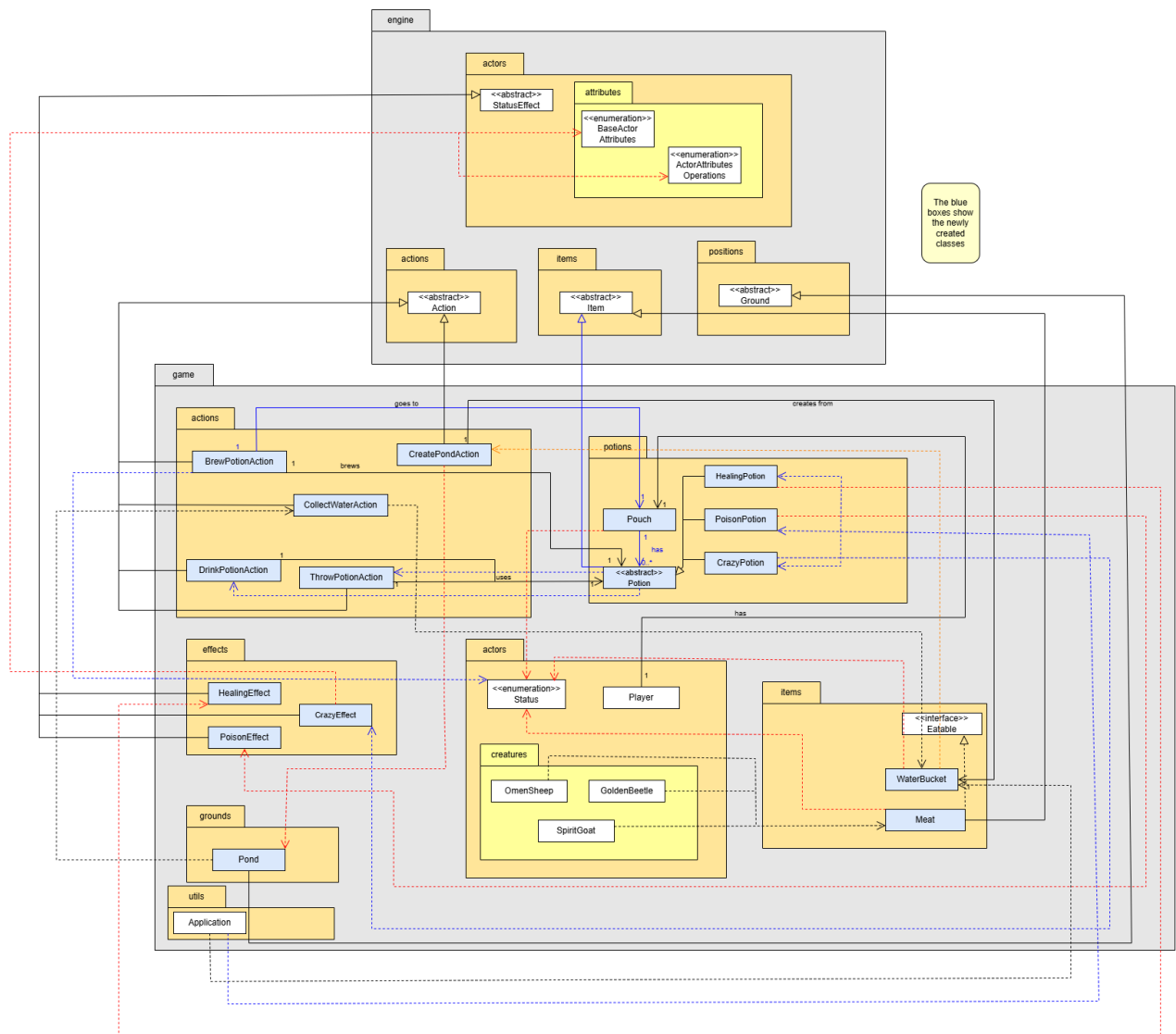
Design 1: Use Separate meat class to represent each creature meat along with their specified Status

Pros	Cons
Interaction logic (like EatAction) can be embedded directly in the Player or game loop, making quick prototyping easier.	Core behavior is dispersed across classes like Player, reducing cohesion and increasing maintenance difficulty.
New meat types can be added quickly without refactoring other classes.	As the system grows, managing individual meat subclasses becomes redundant and harder to maintain.

Design 2: Use a single Meat class with Status passed in as parameter

Pros	Cons
New meat can be added through new Status or Action classes, avoiding changes to existing logic (Open/Closed Principle).	Requires understanding of capabilities (Status) and how they're used for behavior control.
Only one Meat class is needed, avoiding unnecessary subclasses.	
Common logic for different meats (e.g., portable, edible) is centralized.	

UML REQ4



The diagram above illustrates the final design of **Requirement 4**, which applies **Design 1** for the **Potion Effect and Brewing System**, and **Design 2** for the **Meat Ingredients Implementation**.

Having separate Effect and StatusEffect classes is good object-oriented design because it cleanly separates **instantaneous effects** (like HealEffect) from **ongoing, time-based effects** (like HealingEffect). This distinction improves **clarity**: developers can easily tell whether an effect happens once or persists over time. It also enhances **flexibility and extensibility**, allowing you to add new types of effects without overcomplicating a single class.

The Effect interface handles one-off logic through its `apply()` method, making it suitable for consumable items or immediate game events. Meanwhile, `StatusEffect` defines behavior via a `tick()` method, designed to be called each turn—ideal for effects like poisoning or regeneration. Keeping these concerns separate aligns with the **Single Responsibility Principle**, making the codebase easier to maintain and debug. Additionally, it allows the engine to process each type efficiently and correctly within their respective game loops.

For the meat system, a **single Meat class** is used, with name and Status passed into the constructor. This approach provides flexibility while avoiding deep abstraction hierarchies. Variants of meat with different brewing statuses can be created easily without needing separate subclasses. This design is both **highly extensible** and supports **dependency injection**, as behaviour is determined by the passed-in status rather than hard-coded logic.

The design adheres strongly to the **Single Responsibility Principle (SRP)**. Each `StatusEffect` subclass encapsulates specific effect behaviour. Potion subclasses manage potion-specific data such as name, effects, and recipes. The Pouch class is responsible solely for tracking materials and recipes, while `BrewPotionAction` handles the brewing operation independently. Each class has a focused, well-defined responsibility.

We also comply with the **Open/Closed Principle (OCP)**. New potions or effects can be introduced by simply creating new subclasses without modifying existing code. Adding a new brewing recipe requires only its initialization in the Application class and registration with the Pouch—existing classes remain untouched.

The **Liskov Substitution Principle (LSP)** is respected throughout the design. Any subclass of `StatusEffect` (e.g., `PoisonEffect`, `SpeedEffect`) can be used wherever a `StatusEffect` is expected. Similarly, item-interaction Action subclasses can be passed via `allowableActions` and function correctly without altering the consumer's logic.

The **Interface Segregation Principle (ISP)** is applied by keeping interfaces lean and focused. The `StatusEffect` interface includes only essential methods like `tick()` and `toString()`. The Pouch class exposes only the methods relevant to material checking and consumption, while `BrewPotionAction` implements only what's necessary for the brewing process—no class is burdened with irrelevant responsibilities.

Dependency Inversion Principle (DIP) is reinforced by depending on abstractions. For instance, the Player interacts with items through abstract Action classes, not their concrete implementations. The brewing system interacts with the abstract `BrewPotionAction` and Pouch interfaces rather than specific recipes. Likewise, the potion system relies on the `StatusEffect` abstraction instead of being tightly coupled to concrete subclasses.

The design follows the **DRY (Don't Repeat Yourself)** principle by centralizing shared logic. All potion effects are implemented once in `StatusEffect` subclasses and reused across multiple potions. Material checking is implemented in `Pouch` and reused by `BrewPotionAction`.

From a connascence perspective, the design exhibits mostly **Connascence of Name (CoN)** and **Connascence of Type (CoT)**. Classes like `Potion` and `StatusEffect` agree on method names and expected types (e.g., `tick()`), which are manageable through strong interface contracts. More fragile forms like **Connascence of Position (CoP)** or **Meaning (CoM)** are avoided by clearly naming parameters and encapsulating logic within isolated components.

Overall, the system is built for **long-term maintainability** and **modular growth**. Adding new features—such as a new potion effect (`RegenerationEffect`) or a new creature that drops meat—requires no modification to existing logic. Even enabling future NPCs such as wizards to brew potions is seamless: simply inject a `Pouch` during instantiation via the `Application` class.

This architecture demonstrates strong alignment with **SOLID principles**, resulting in a clear separation of concerns, low coupling, and high cohesion. It lays a robust, extensible foundation that allows the system to evolve organically without unnecessary complexity or refactoring.

Sequence Diagram REQ4

